

LAPORAN PRAKTIKUM PENGEMBANGAN APLIKASI MOBILE



Dosen Pengampu :

Muhammad Aminul Akbar, S.Kom., M.T.

Disusun Oleh :

Nama : Nada Halwa Yuanita

NIM : 245150407111021

PROGRAM STUDI SISTEM INFORMASI

JURUSAN SISTEM INFORMASI

FAKULTAS ILMU KOMPUTER

UNIVERSITAS BRAWIJAYA

MALANG

2026

MODUL 3

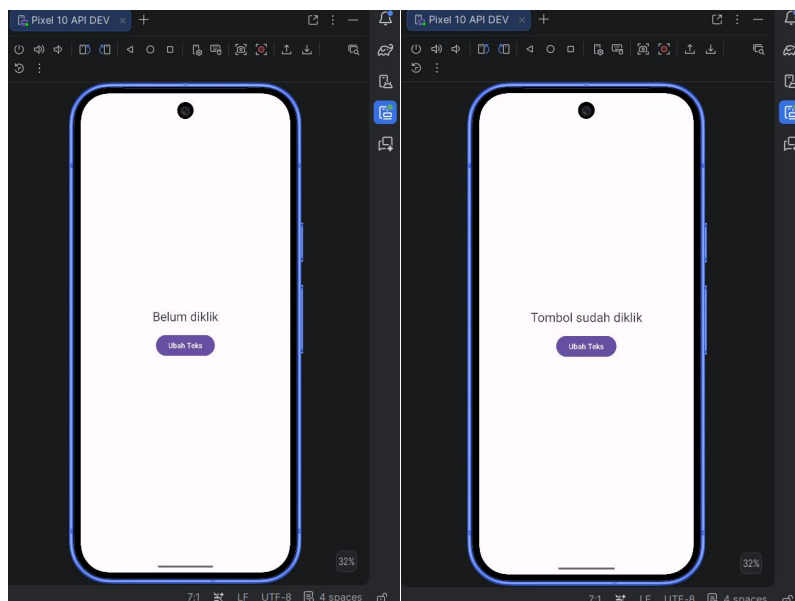
EVENT, STATE, IMPERATIF-DEKLARATIF, DAN IMPLISIT INTENT PADA JETPACK COMPOSE

A. Percobaan 1 Perbandingan Imperatif dan Deklaratif dalam Perubahan Teks

Pada percobaan ini dibuat dua proyek Android Studio yang terpisah, yaitu ImperativeXmlApp yang dibangun menggunakan pendekatan XML/View secara imperatif, dan DeclarativeComposeApp yang dibangun menggunakan Jetpack Compose secara deklaratif. Kedua proyek dirancang agar menghasilkan tampilan akhir yang sama persis, yakni teks awal "Belum diklik" yang berubah menjadi "Tombol sudah diklik" setelah tombol "Ubah Teks" ditekan, sehingga perbedaan yang sesungguhnya ingin diamati bukan pada hasil akhirnya, melainkan pada bagaimana masing-masing pendekatan memperbarui tampilan antarmuka.

1. Jalankan kedua aplikasi dan ambil screenshot kondisi sebelum serta sesudah tombol diklik.

Jawaban: Kedua aplikasi telah dijalankan pada emulator Pixel 10 dan menunjukkan perilaku yang sesuai dengan yang diharapkan, yaitu teks berubah dari "Belum diklik" menjadi "Tombol sudah diklik" setelah tombol ditekan, dengan tampilan akhir yang identik antara Project A dan Project B.



2. Pada project XML/View, objek apa yang diubah secara langsung ketika tombol diklik? Tunjukkan baris kodenya.

Jawaban: Objek yang diubah secara langsung adalah instance TextView bernama tvMessage, yaitu pada baris kode tvMessage.text = "Tombol sudah diklik" yang berada di dalam blok setOnClickListener milik btnChange. Baris ini secara eksplisit mengakses properti text dari objek TextView yang sebelumnya sudah diperoleh melalui findViewById(), sehingga perubahan tampilan terjadi karena objek View itu sendiri yang dimodifikasi secara langsung oleh kode program.

3. Pada project Compose, nilai apa yang diubah ketika tombol diklik? Tunjukkan baris kodenya.

Jawaban: Nilai yang diubah bukanlah objek antarmuka seperti Text, melainkan variabel state bernama message, tepatnya pada baris message = "Tombol sudah diklik" di dalam blok onClick milik Button. Variabel message ini dideklarasikan menggunakan var message by remember { mutableStateOf("Belum diklik") }, sehingga ketika nilainya diubah, Compose yang akan menangani pembaruan tampilan, bukan program yang mengubah komponen Text secara langsung.

4. Mengapa pada Compose tidak terdapat perintah seperti tvMessage.text atau setText()?

Jawaban: ini terjadi karena pada Jetpack Compose, pengembang tidak pernah memegang referensi langsung terhadap komponen antarmuka yang ditampilkan. Baris Text(text = message) hanya mendeklarasikan bahwa komponen tersebut akan menampilkan apa pun nilai yang sedang dimiliki oleh message pada saat itu, tanpa menyebutkan instruksi untuk mengubah komponen secara manual. Compose sendiri yang bertanggung jawab menentukan bagian antarmuka mana yang perlu digambar ulang ketika state yang menjadi sumber datanya berubah, sehingga konsep seperti setText() menjadi tidak relevan dalam paradigma deklaratif ini.

5. Jelaskan alur perubahan pada XML/View dengan pola: event -> View -> perubahan tampilan.

Jawaban: Alurnya dimulai ketika pengguna menekan tombol sehingga event onClick pada listener btnChange terpicu. Selanjutnya, kode program mengakses objek View yang relevan, dalam hal ini tvMessage, melalui referensi yang telah diperoleh sebelumnya

menggunakan `findViewById()`. Setelah objek tersebut diperoleh, properti text pada objek tersebut diubah secara langsung, dan perubahan tampilan terjadi karena objek View itu sendiri yang telah dimodifikasi, bukan karena adanya mekanisme pembaruan otomatis dari sistem.

6. Jelaskan alur perubahan pada Compose dengan pola: event -> state -> recomposition -> perubahan tampilan.

Jawaban: Alurnya dimulai ketika pengguna menekan tombol sehingga event `onClick` terpicu. Event tersebut kemudian mengubah nilai state, yaitu variabel `message`, bukan mengubah komponen antarmuka secara langsung. Compose mendeteksi bahwa state yang diamati telah berubah, sehingga secara otomatis memicu proses recomposition pada bagian composable yang bergantung pada state tersebut, dalam hal ini fungsi `Text` yang membaca nilai `message`. Melalui proses recomposition inilah tampilan pada layar akhirnya diperbarui sesuai dengan nilai state yang terbaru.

7. Apa yang terjadi jika message diganti menjadi variabel biasa: `var message = "Belum diklik"`? Jelaskan penyebabnya.

Jawaban: Apabila `message` diubah menjadi variabel biasa tanpa dibungkus oleh `remember` dan `mutableStateOf`, maka penekanan tombol tetap dapat mengubah nilai variabel tersebut secara internal, namun tampilan pada layar tidak akan mengalami perubahan apa pun. Penyebabnya adalah Compose hanya dapat mendeteksi dan merespons perubahan pada objek yang bersifat observable, yaitu objek yang diciptakan melalui `mutableStateOf`. Variabel biasa tidak memiliki mekanisme untuk memberi tahu Compose bahwa nilainya telah berubah, sehingga tidak ada recomposition yang terpicu. Selain itu, tanpa `remember`, nilai `message` juga akan kembali ke kondisi awal setiap kali terjadi recomposition pada fungsi composable tersebut, karena `remember`-lah yang bertugas mempertahankan nilai suatu state selama composable masih berada dalam composition.

8. Tuliskan kesimpulan perbedaan cara berpikir imperatif dan deklaratif menggunakan kalimat Anda sendiri.

Jawaban: Berdasarkan hasil percobaan ini, dapat disimpulkan bahwa pendekatan imperatif dan deklaratif memiliki cara berpikir yang mendasar berbeda dalam mengelola tampilan antarmuka. Pada pendekatan imperatif, pengembang bertindak seolah memberi

perintah langsung kepada komponen antarmuka, yaitu dengan cara mencari objek yang dituju kemudian mengubah propertinya secara eksplisit, sehingga tanggung jawab untuk menyinkronkan tampilan dengan data sepenuhnya berada di tangan pengembang. Sebaliknya, pada pendekatan deklaratif, pengembang cukup mendeskripsikan bagaimana tampilan seharusnya terlihat berdasarkan kondisi data atau state saat ini, dan ketika data tersebut berubah, framework Compose yang akan menentukan serta melaksanakan pembaruan tampilan yang diperlukan melalui mekanisme recomposition. Dengan demikian, perbedaan utamanya bukan terletak pada hasil akhir yang ditampilkan kepada pengguna, melainkan pada siapa yang memegang kendali dan tanggung jawab dalam menjaga kesesuaian antara data dan tampilan.

B. Percobaan 2 Event Handling State

Pada percobaan ini dibuat satu proyek baru bernama EventStateApp menggunakan Jetpack Compose, yang dikembangkan secara bertahap melalui tiga tahap pengujian, yaitu penggunaan variabel biasa, penggunaan state melalui remember dan mutableStateOf, serta penambahan state baru berupa input teks melalui OutlinedTextField.

Tahap 1 Counter Menggunakan Variabel Biasa (var count = 0)

2. Apa yang muncul di LogCat setiap kali tombol "Tambah" diklik?

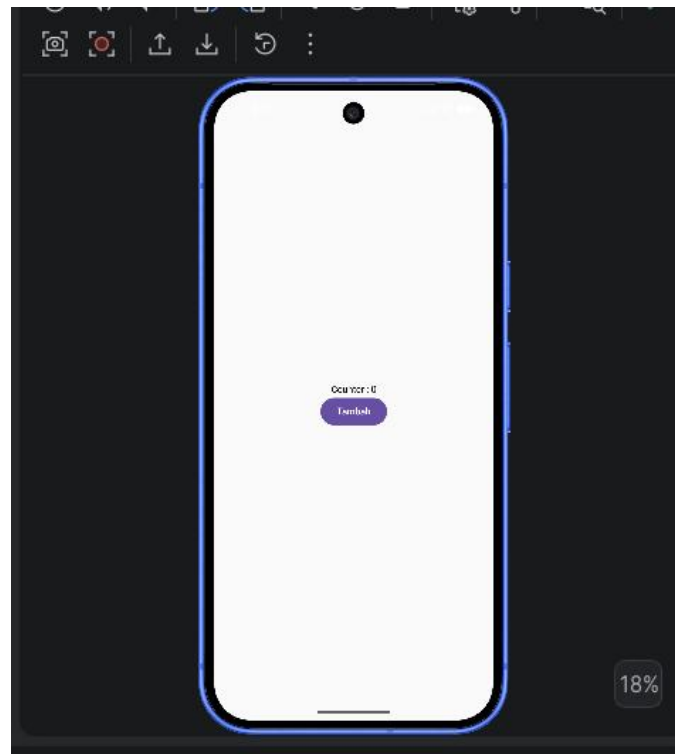
Jawaban: Setiap kali tombol "Tambah" ditekan, LogCat menampilkan baris log berupa "Counter berubah : N", dengan nilai N yang terus bertambah satu setiap kali tombol tersebut ditekan, sebagaimana terlihat pada hasil pengujian yang menunjukkan log "Counter berubah : 1", "Counter berubah : 2", dan "Counter berubah : 3" secara berurutan.

3. Apa yang terjadi pada tampilan aplikasi setiap kali tombol "Tambah" diklik?

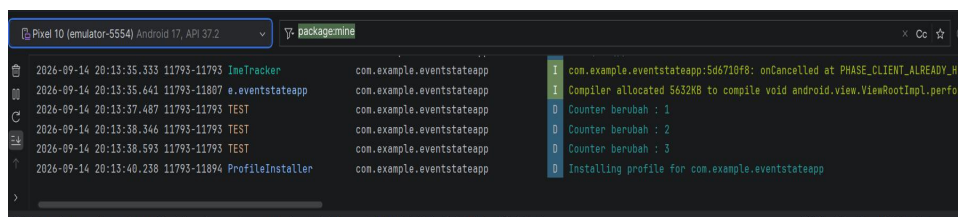
Jawaban: Tampilan aplikasi tidak mengalami perubahan sama sekali. Teks "Counter : 0" yang tertampil di layar tetap bertahan pada angka nol meskipun tombol "Tambah" telah ditekan berulang kali, sekalipun LogCat menunjukkan bahwa nilai count sesungguhnya terus bertambah di balik layar.

4. Apa yang menyebabkan hal tersebut terjadi?

Jawaban: Penyebabnya adalah variabel count pada tahap ini dideklarasikan sebagai variabel Kotlin biasa (var count = 0), bukan sebagai state yang dapat diamati oleh Compose. Nilai variabel tersebut memang berubah secara nyata di dalam memori, sehingga proses logika program seperti pencatatan log tetap berjalan normal, namun karena Compose tidak memiliki mekanisme untuk mendeteksi perubahan pada variabel biasa, tidak ada proses recomposition yang terpicu, sehingga fungsi Text yang menampilkan "Counter : \$count" tetap menampilkan nilai dari komposisi pertama kalinya, yaitu nol, selamanya.



Gambar 1. Tampilan aplikasi tetap menunjukkan "Counter : 0" meskipun tombol telah ditekan berulang kali



Gambar 2. LogCat menunjukkan nilai count yang terus bertambah meskipun tampilan tidak berubah

Tahap 2 Counter Menggunakan State (var count by remember { mutableStateOf(0) })

9. Apa yang muncul di LogCat setiap kali tombol "Tambah" diklik?

Jawaban: Pola yang muncul di LogCat tetap sama seperti sebelumnya, yaitu baris log "Counter berubah : N" yang nilainya terus bertambah setiap kali tombol "Tambah" ditekan.

10. Apa yang terjadi pada tampilan aplikasi setiap kali tombol "Tambah" diklik?

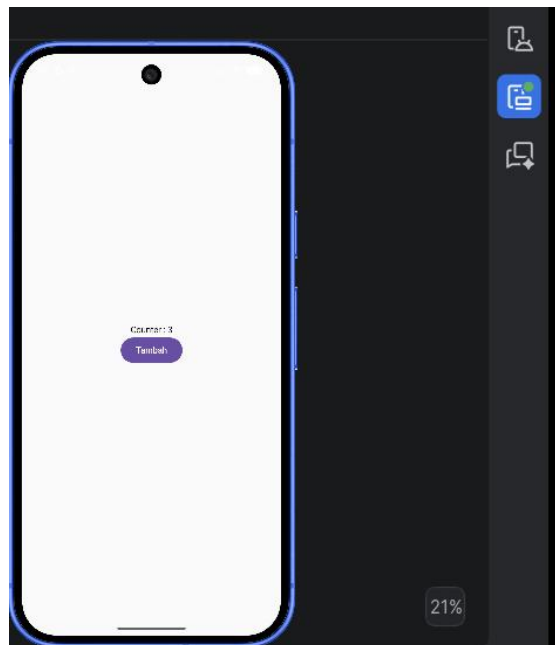
Jawaban: Berbeda dengan tahap sebelumnya, kali ini tampilan aplikasi ikut berubah secara langsung. Teks "Counter : N" pada layar bertambah nilainya setiap kali tombol ditekan, sejalan dan sinkron dengan nilai yang tercatat pada LogCat.

11. Apa yang menyebabkan hal tersebut terjadi dan berbeda dengan percobaan sebelumnya?

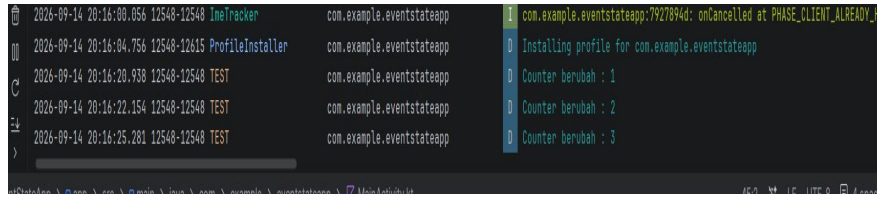
Jawaban: Penyebab utamanya adalah perubahan deklarasi count dari variabel biasa menjadi var count by remember { mutableStateOf(0) }, yang menjadikan count sebagai observable state yang dapat dipantau oleh Compose. Setiap kali nilai count berubah, Compose mendeteksi perubahan tersebut dan secara otomatis memicu recomposition pada composables yang bergantung pada count, yaitu fungsi Text yang menampilkannya, sehingga tampilan pada layar ikut diperbarui. Inilah yang membedakannya dari percobaan sebelumnya, di mana variabel biasa tidak memiliki kemampuan untuk memberi tahu Compose bahwa nilainya telah berubah.

12. Apa yang dimaksud recomposition pada Compose?

Jawaban: Recomposition adalah proses yang dilakukan oleh Jetpack Compose untuk menjalankan ulang fungsi composables yang bergantung pada suatu state, guna menghasilkan tampilan antarmuka yang baru sesuai dengan nilai state yang terkini. Compose dirancang agar proses ini berjalan secara efisien, yaitu hanya bagian antarmuka yang benar-benar terpengaruh oleh perubahan state saja yang akan digambar ulang, bukan keseluruhan tampilan aplikasi dari awal.



Gambar 3. Tampilan aplikasi menunjukkan "Counter : 3" setelah tombol ditekan tiga kali, sesuai dengan nilai pada LogCat



Gambar 4. LogCat menunjukkan nilai count yang sejalan dengan tampilan pada layar

Tahap 3 Penambahan State Baru Berupa OutlinedTextField

15. Apa yang muncul di LogCat setiap kali menginputkan huruf di Text Field?

Jawaban: Setiap kali satu huruf diketikkan pada Text Field, LogCat menampilkan baris log "Text berubah : <isi terkini>", dengan isi teks yang terus bertambah sesuai karakter yang baru saja diketikkan, sebagaimana terlihat pada hasil pengujian yang menunjukkan log berturut-turut seperti "Text berubah : Nada Halwa Y", "Text berubah : Nada Halwa Yu", hingga rangkaian huruf berikutnya.

16. Apa yang terjadi pada tampilan aplikasi setiap kali menginputkan huruf di Text Field?

Jawaban: Tampilan pada kolom Text Field langsung menunjukkan karakter yang baru saja diketikkan secara real-time, sehingga isi kolom tersebut selalu sesuai dengan apa yang sedang diketikkan oleh pengguna pada saat itu juga.

17. Apa yang menyebabkan hal tersebut terjadi?

Jawaban: Penyebabnya sejalan dengan konsep pada tahap sebelumnya, yaitu variabel text dideklarasikan sebagai `var text by remember { mutableStateOf("") }`, sehingga berstatus sebagai observable state. Setiap kali terjadi perubahan pada kolom input, fungsi `onValueChange` dijalankan dan mengubah nilai text, yang kemudian terdeteksi oleh Compose dan memicu recomposition pada `OutlinedTextField` beserta bagian lain yang bergantung pada nilai text tersebut, sehingga tampilan kolom input selalu diperbarui mengikuti karakter yang baru diketikkan.

18. Sebutkan state apa saja yang ada pada program yang terakhir dibuat?

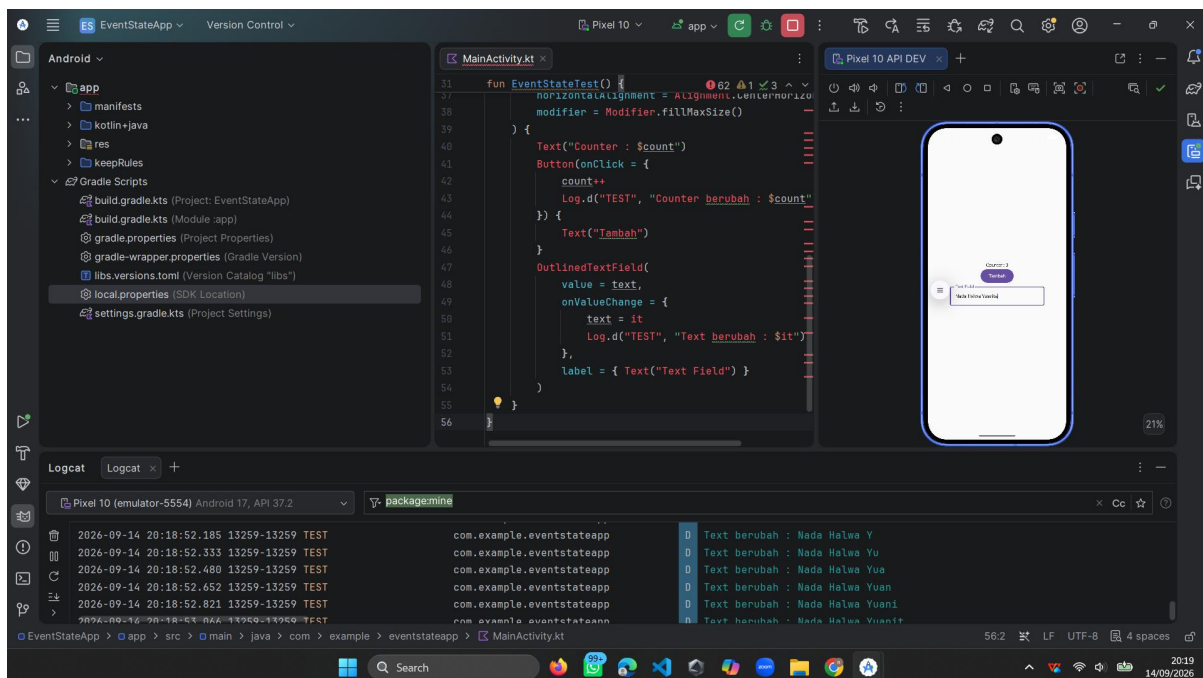
Jawaban: Terdapat dua buah state pada program ini, yaitu count yang bertipe data `Int` dan berfungsi untuk mencatat jumlah penekanan tombol "Tambah", serta text yang bertipe data `String` dan berfungsi untuk menyimpan isi masukan pengguna pada kolom `OutlinedTextField`.

19. Sebutkan event apa saja yang ada pada program yang terakhir dibuat?

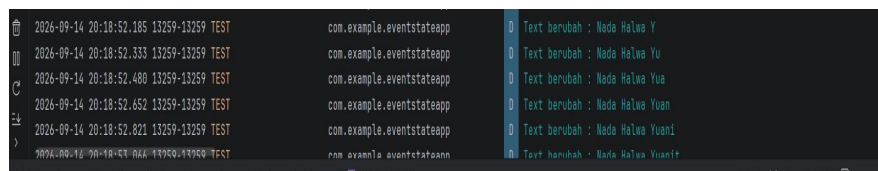
Jawaban: Terdapat dua buah event pada program ini, yaitu onClick yang melekat pada komponen Button, dan onValueChanged yang melekat pada komponen OutlinedTextField.

20. Kapan event-event tersebut dijalankan?

Jawaban: Event onClick dijalankan setiap kali pengguna menekan atau mengklik tombol "Tambah" pada layar. Sementara itu, event onValueChanged dijalankan setiap kali terjadi perubahan pada nilai OutlinedTextField, yaitu setiap kali ada perubahan karakter di dalamnya, baik karena penambahan huruf, penghapusan huruf, maupun penempelan (paste) teks, sehingga event ini dapat terpicu berkali-kali selama proses pengetikan berlangsung, bukan hanya sekali di akhir.



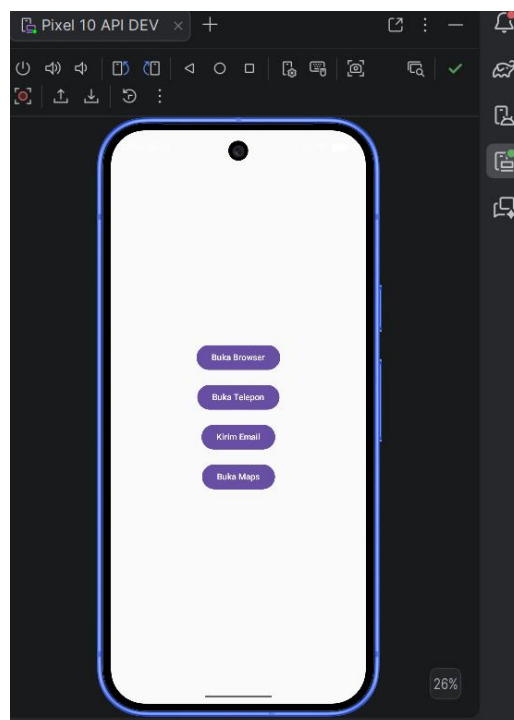
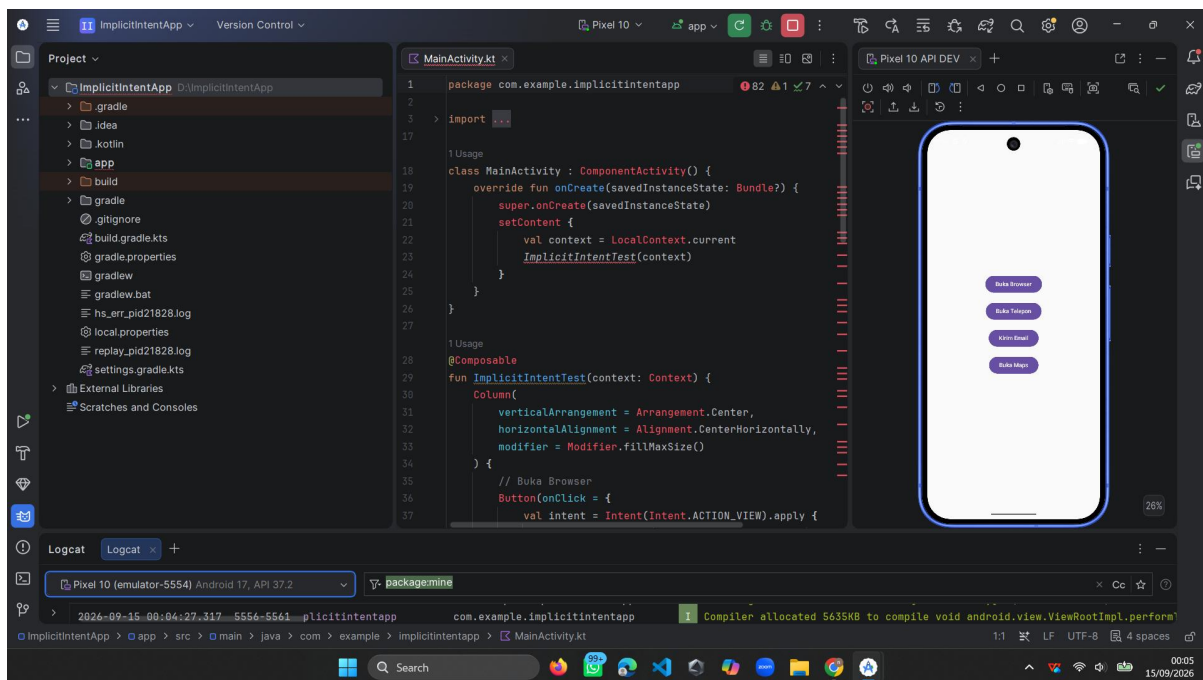
Gambar 5. Kolom Text Field menampilkan teks yang sedang diketik secara real-time, sementara nilai counter dari tahap sebelumnya tetap tersimpan



Gambar 6. LogCat menunjukkan isi teks yang bertambah satu karakter setiap kali pengguna mengetik

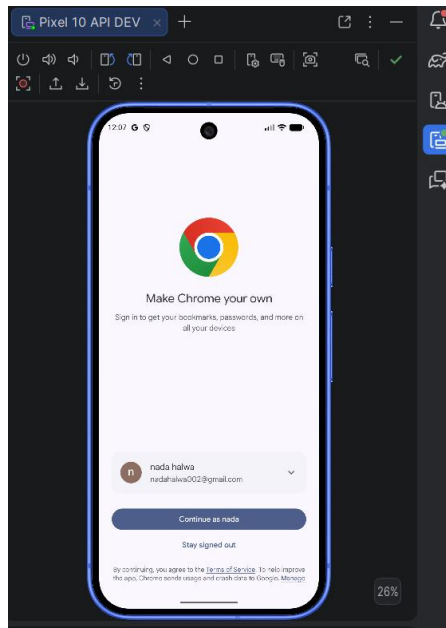
C. Percobaan 3 Implisit Intent

Pada percobaan ini dibuat proyek baru bernama ImplicitIntentApp menggunakan Jetpack Compose, yang berisi empat buah tombol untuk menguji penggunaan implicit intent, yaitu untuk membuka peramban (browser), membuka layar panggilan telepon, mengirim email, dan membuka aplikasi peta (Maps). Seluruh tombol telah diuji coba pada emulator dan menghasilkan perilaku yang sesuai dengan fungsi masing-masing action yang digunakan.

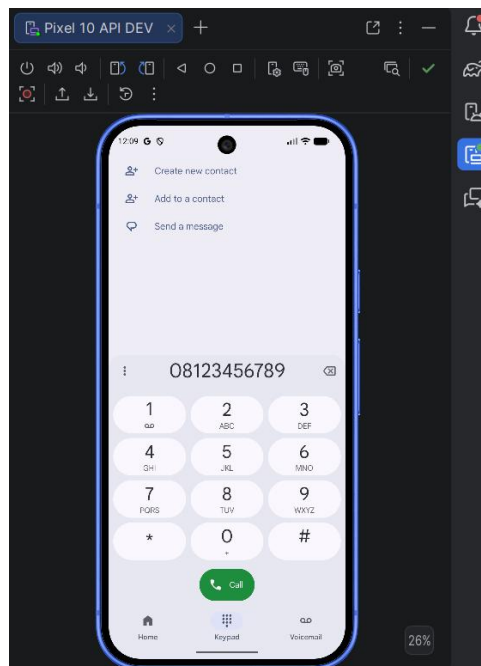


Gambar 7. Tampilan aplikasi ImplicitIntentApp dengan empat tombol pengujian implicit intent

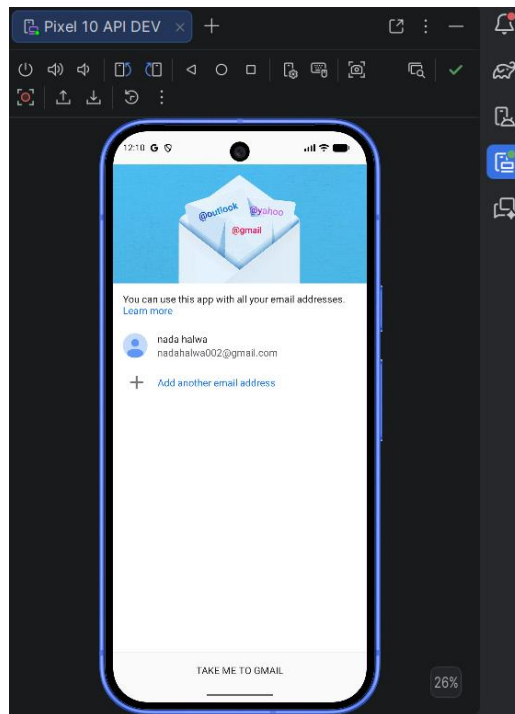
Hasil pengujian menunjukkan bahwa tombol "Buka Browser" berhasil membuka aplikasi Chrome menampilkan halaman google.com, tombol "Buka Telepon" berhasil membuka layar dial-pad dengan nomor 08123456789 yang telah terisi secara otomatis, tombol "Kirim Email" menampilkan chooser aplikasi email yang mengarahkan pengguna ke aplikasi Gmail, dan tombol "Buka Maps" berhasil membuka aplikasi Google Maps yang menampilkan peta sesuai koordinat yang telah ditentukan dalam kode program.



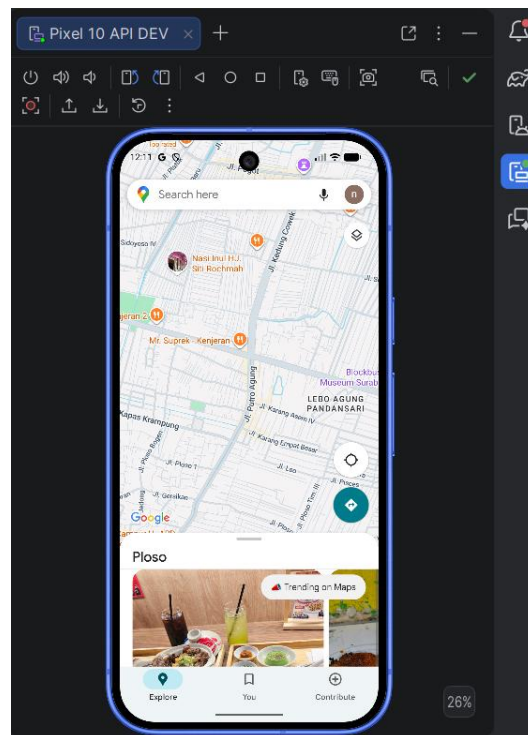
Gambar 8. Hasil pengujian tombol "Buka Browser"



Gambar 9. Hasil pengujian tombol "Buka Telepon"



Gambar 10. Hasil pengujian tombol "Kirim Email"



Gambar 11. Hasil pengujian tombol "Buka Maps"

11. Apa perbedaan fungsi dari masing-masing action intent tersebut (Intent.ACTION_VIEW, Intent.ACTION_DIAL, Intent.ACTION_SENDTO)?

Jawaban: Intent.ACTION_VIEW digunakan untuk menampilkan suatu konten kepada pengguna berdasarkan data yang diberikan, di mana sistem Android akan menentukan

sendiri aplikasi yang paling sesuai untuk menampilkannya, misalnya membuka peramban untuk data berupa tautan web atau membuka Maps untuk data berupa koordinat lokasi. `Intent.ACTION_DIAL` digunakan secara khusus untuk membuka layar dial-pad telepon dengan nomor yang sudah terisi otomatis, tanpa langsung melakukan panggilan, sehingga pengguna masih perlu menekan tombol panggil secara manual. Adapun `Intent.ACTION_SENDTO` digunakan untuk mengarahkan pengiriman sesuatu, dalam kasus ini email, ke tujuan tertentu berdasarkan skema data yang diberikan, di mana sistem akan mencari aplikasi yang mampu menangani proses pengiriman tersebut.

16. Apa fungsi data pada intent?

Jawaban: Data pada intent, yang berupa objek Uri, berfungsi untuk memberi tahu sistem Android mengenai konten apa yang hendak diproses beserta skema atau protokol yang digunakan. Skema pada Uri inilah yang menjadi acuan bagi sistem untuk menentukan kategori aplikasi yang relevan, misalnya skema `https://` untuk konten web, `tel:` untuk nomor telepon, `mailto:` untuk alamat email, dan `geo:` untuk koordinat lokasi geografis.

17. Apa hubungan Action dan Data pada pilihan aplikasi yang dimunculkan oleh sistem Android?

Jawaban: Action menyatakan jenis tindakan yang ingin dilakukan, seperti menampilkan, memanggil, atau mengirim, sedangkan Data menyatakan objek atau konten yang menjadi sasaran tindakan tersebut beserta format atau skemanya. Kombinasi antara Action dan Data inilah yang digunakan oleh sistem Android melalui mekanisme intent resolution untuk mencocokkan permintaan tersebut dengan intent-filter yang telah didaftarkan oleh masing-masing aplikasi yang terpasang pada perangkat. Sebagai contoh, kombinasi `ACTION_VIEW` dengan data berskema `https` akan diarahkan ke aplikasi peramban karena aplikasi tersebut mendaftarkan diri sebagai penangan skema `https`, sementara kombinasi `ACTION_VIEW` dengan data berskema `geo` akan diarahkan ke aplikasi peta. Karakteristik inilah yang menjadikan intent jenis ini disebut sebagai implicit intent, sebab pengembang tidak perlu menyebutkan nama aplikasi tujuan secara eksplisit, melainkan cukup menentukan Action dan Data yang sesuai, dan sistem yang akan menentukan aplikasi penanganannya.

D. Kesimpulan

Melalui rangkaian tiga percobaan pada modul ini, dapat disimpulkan bahwa Jetpack Compose menerapkan paradigma pemrograman antarmuka yang bersifat deklaratif dan berbasis state, yang secara mendasar berbeda dari pendekatan imperatif pada sistem View/XML konvensional. Perubahan tampilan pada Compose hanya dapat terjadi apabila data yang menjadi sumbernya dideklarasikan sebagai observable state melalui `mutableStateOf`, dan dipertahankan nilainya antar-recomposition melalui `remember`. Event seperti `onClick` dan `onValueChange` berperan sebagai pemicu perubahan pada state, bukan sebagai perintah langsung terhadap komponen antarmuka, sehingga tanggung jawab pembaruan tampilan sepenuhnya diserahkan kepada mekanisme recomposition milik Compose. Selain itu, melalui percobaan `implicit intent`, dapat dipahami bahwa Android menyediakan mekanisme komunikasi antaraplikasi yang fleksibel, di mana kombinasi Action dan Data yang tepat memungkinkan suatu aplikasi memanfaatkan fungsi dari aplikasi lain tanpa perlu mengetahui secara spesifik aplikasi mana yang akan menanganinya.